



Ch 2: Knowledge Representation and Reasoning

Artificial Intelligence

Dr. Udaya R. Dhungana
Assistant Professor
Pokhara University, Nepal

Former Guest Faculty, Darmstadt University of Applied Sciences, Germany



Overview

Ch 2: Knowledge Representation and Reasoning (7 hrs)

1. First-Order Logic (FOL) and reasoning
2. Probabilistic reasoning
 - 2.1.1. Hidden Markov Models
 - 2.1.2. Dynamic Bayesian Networks
3. Ontological Engineering
4. Semantic Networks
5. Description Logic
6. Fuzzy Logic-
 - 6.1.1. Fuzzy Inference Systems
 - 6.1.1.1. Mamdani
 - 6.1.1.2. Sugeno
 - 6.1.1.3. Tsukamoto
 - 6.1.2. Defuzzification techniques

1. Introduction to Ontology
2. Why Ontology
3. Application of Ontology
4. Ontology Engineering
5. Ontology Development Life Cycle
6. Ontology Languages

ONTOLOGY

Ontology

- Ontology is a formal *representation of knowledge* as
 - a set of *concepts* within a domain and
 - the *relationships* between those *concepts*.
- Ontologies provide a *structured framework* for
 - modeling knowledge
 - enabling reasoning
- The word *ontology* refers to the study of *being* or *existence*
 - ***In Computer Science:*** **Ontology is used to describe models of reality** that machines can understand, manipulate and reason



Ontology

- Ontology is a formal *representation of knowledge* as
 - a set of *concepts* within a domain and
 - the *relationships* between those *concepts*.

An ontology is a formal specification of the knowledge of a particular domain, making it amenable to machine processing



Ontology

- An ontology is a formal specification of the knowledge of a particular domain, making it amenable to machine processing. Such a specification consists of
 - **terminology** (or vocabulary) of the domain
 - semantic **relationships** between terms
- **Example:** in the university domain
 - **terms:** student, PhD student, professor, course, teaches, takes, supervises
 - **relationships** between terms:
 - every PhD student is a student
 - cannot be both student and professor
 - every course is taught by some professor



Why Ontology?

To **standardize the terminology** of an application domain

- **meaning of terms is constrained**, so less misunderstandings
- by adopting a common vocabulary, **easy to share information**

To support **automated reasoning**

- **uncover errors in modelling (debugging)**
- **exploit knowledge in the ontology during query answering**, to get back a **more complete set of answers** to queries

To present an **intuitive and unified view of data**

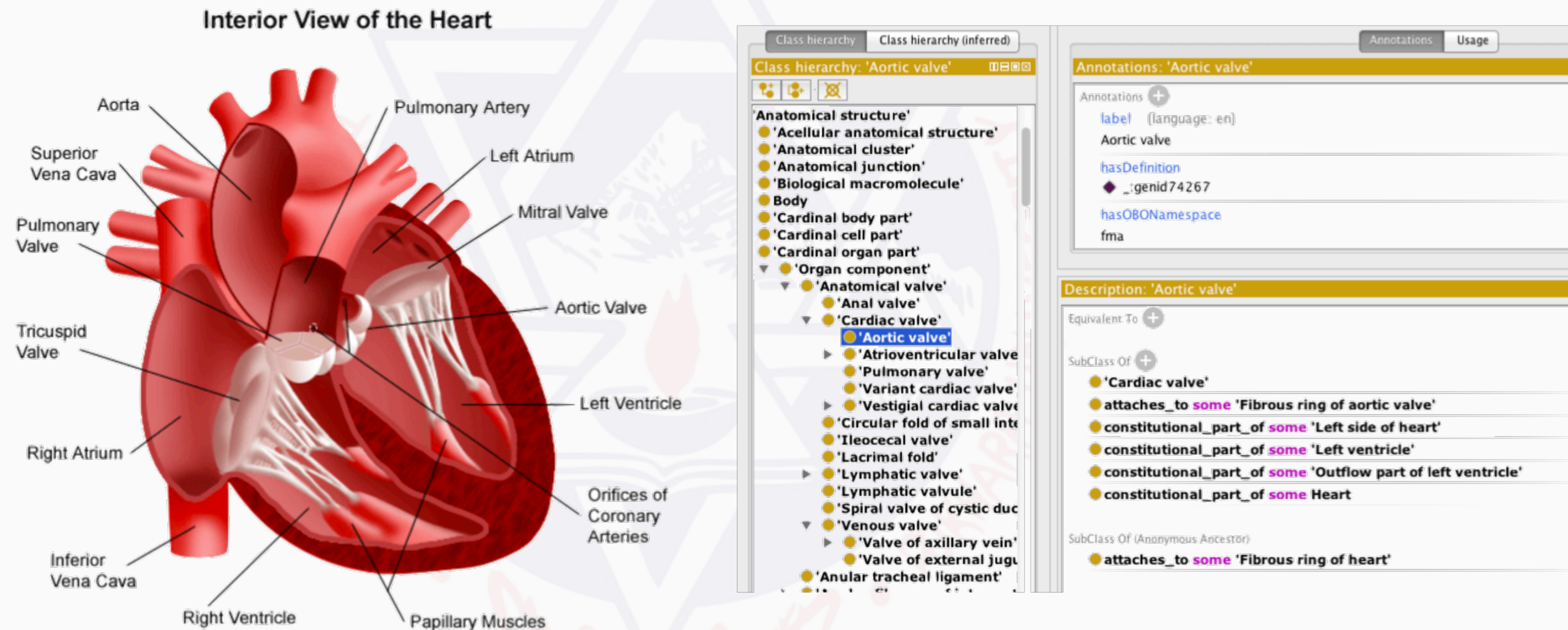
- ontology can be used to **enrich the data vocabulary**, making it **easier for users to formulate their queries**
- especially useful when **integrating multiple data sources**



Applications of Ontology: Medicine

General medical ontologies: **SNOMED CT**, GALEN

Specialized ontologies: FMA (anatomy), NCI (cancer), ...



Querying & exchanging medical records (find patients for medical trials)

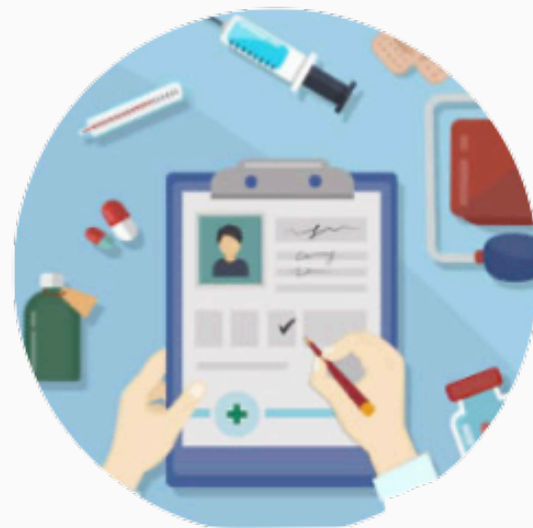
Supports analysis of health data for medical research



General Medical Ontology: SNOMED CT

Large-scale comprehensive medical ontology

- more than **350,000 terms** on all aspects of clinical healthcare:
 - symptoms, diagnosis, medical procedures, body structures, organisms, substances, pharmaceutical products, etc.



Widely adopted standard for healthcare terminology

- in use in **> 80 countries** (including U.S., Canada, UK)
- utilized in **health IT** (e.g. IBM Watson Health, Babylon Health)



Ontology Engineering

- *Ontology engineering* refers to the set of activities concerned with the development, maintenance, and application of ontologies.
- *Ontology* provides
 - a formal model of a domain,
- *Ontology engineering* provides
 - the methodologies, tools, languages, and processes for building and managing ontology models effectively

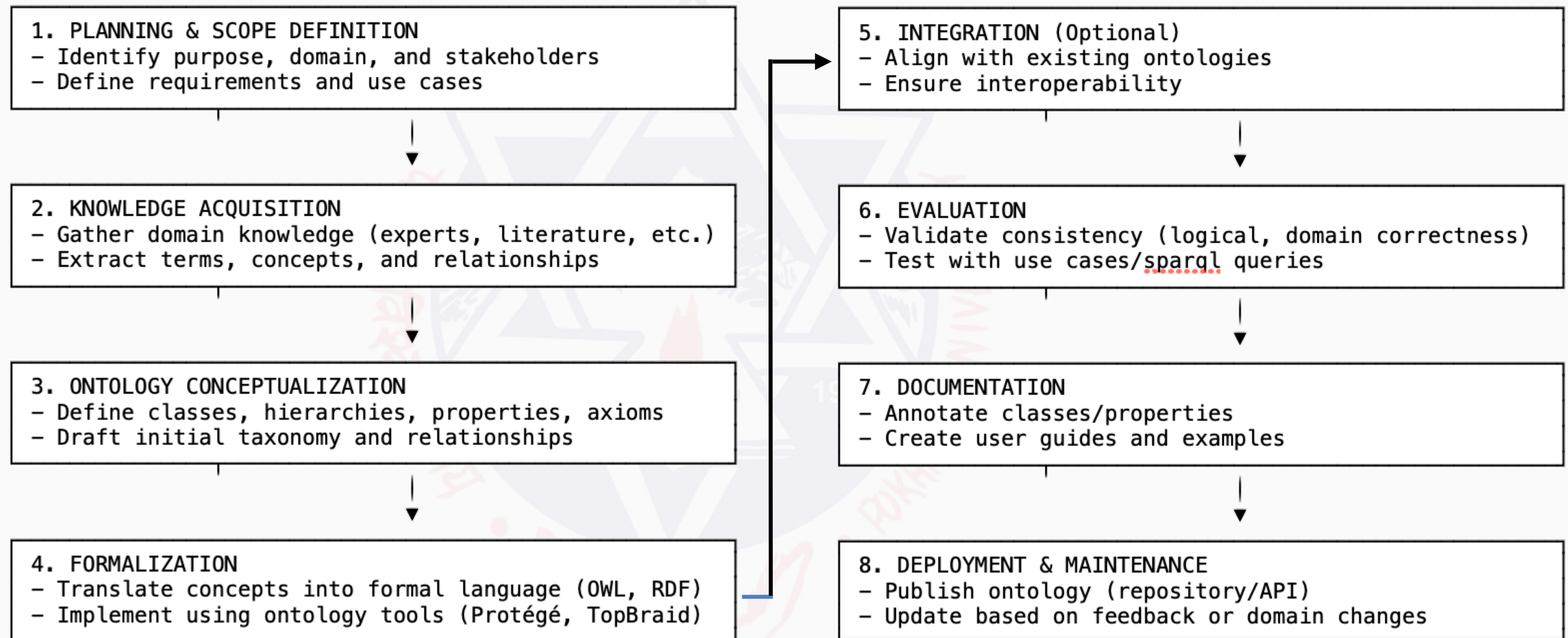


Ontology Engineering

- ***Ontology engineering*** is therefore
 - the systematic process of designing, building, and managing ontologies
 - concerned with capturing knowledge about a domain and formalizing it in a reusable, shareable, and machine-interpretable way
 - A core discipline in Semantic Web technologies, enterprise knowledge modeling, and AI systems.
- ***Goals:***
 - To produce accurate, consistent, and reusable knowledge models.
 - To ensure ontologies are machine-understandable and interoperable.
 - To support automated reasoning, semantic integration, and knowledge sharing



Ontology Development Life Cycle



Ontology Development Life Cycle

- ***Specification***

- Define domain, scope, purpose, stakeholders.
- Identify intended use (e.g., for reasoning, data integration, NLP).

- ***Conceptualization***

- Identify key concepts, relations, constraints, and rules.
- Organize them into a conceptual model, often in diagrams or mind maps

- ***Formalization***

- Transform the conceptual model into a formal representation using **Description Logic**, RDF, OWL, etc



Ontology Development Life Cycle

- **Implementation**

- Use ontology development tools (e.g., Protégé) to build the formal model.
- Populate ontology with instances (ABox).

- **Evaluation**

- Check for consistency, completeness, and correctness.
- Use automated reasoners and ontology metrics

- **Deployment and Maintenance**

- Make the ontology available through APIs or web services.
- Integrate into applications, e.g., search engines, knowledge graphs
- Update based on feedback or domain knowledge evolves



Ontology Languages?

Which languages can we use to **specify an ontology**?

- What about Natural language?
 - imprecise, ambiguous, not machine-interpretable
- Formal Logic
 - first-order logic (FOL)
 - expressive, well understood, BUT reasoning undecidable •
 - ‘nice’ fragments of FOL (*description logics*, existential rules) •
 - expressivity vs computational complexity tradeoff
- Standardized languages for the Web (OWL, RDFS)
 - closely related and based upon logic



DESCRIPTION LOGIC

Description Logic

- **Description logics** (DLs) are:
 - family of **knowledge representation** languages
 - popular means for specifying **ontologies**
 - range from fairly simple to highly expressive
 - widely used in **semantic web** technologies including **OWL** (Web Ontology Language)
- **Formally:** correspond to **decidable fragments of first-order logic**
 - inherit well-defined semantics
 - convenient variable-free syntax
 - modelling via unary and binary relations



Description Logic

- **Description logics** are notations that are designed to make it easier to describe
 - *definitions* of categories
 - *properties* of categories
- Description logic systems evolved from semantic networks.
- The principal inference tasks for description logics are
 - *subsumption* (checking if one category is a subset of another by comparing their definitions)
 - *classification* (checking whether an object belongs to a category).
 - **consistency** of a category definition (whether the membership criteria are logically satisfiable)



Basic Building Block of DL

- **Concept names (classes)**
 - correspond to sets of entities
 - unary predicates in logic
 - Example: *Parent, Professor, Engineer, Whale, CapitalCity*
- **Role names (properties/relations)**
 - relate two entities (binary relation)
 - binary predicates in logic
 - Example: *fatherOf, teaches, locatedIn, hasHabitat*
- **Individual names (instances)**
 - denote particular entities (constants in logic)
 - Example: *ram, cmp551, pokhara*



Basic Building Block of DL

- **Concept names (classes)**

- correspond to sets of entities
- unary predicates in logic
- Example: *Parent*, *Professor*, *Engineer*, *Whale*, *CapitalCity*

Concepts are exactly the same as Unary Predicates in FOL -
-Person(John)

- **Role names (properties/relations)**

- relate two entities (binary relation)
- binary predicates in logic
- Example: *fatherOf*, *teaches*, *locatedIn*, *hasHabitat*

Roles are exactly the same as Binary Predicates in FOL -
-hasMother(john, mary)

- **Individual names (instances)**

- denote particular entities (constants in logic)
- Example: *ram*, *cmp551*, *pokhara*

Individuals are treated exactly the same as constants in FOL
- john



Common Constructors to Build Complex Descriptions

Combine concepts/classes and roles/properties **using constructors** to **build complex descriptions**

Example: courses taught by Meghyn which are attended by at least two students not from computer science and only by Master's or Phd students

$$\text{Course} \sqcap \exists \text{taughtBy}.\{\text{meghyn}\} \sqcap \geq 2 \text{attendedBy} . (\neg \text{CompSciStudent})$$
$$\forall \text{attendedBy} . (\text{MasterStudent} \sqcup \text{PhDStudent})$$


Conjunction

Notation: $C \sqcap D$

(C, D potentially complex concepts)

Meaning: class of entities that belong to **both C and D**

Female scientist

Female $\sqcap$ Scientist

Spicy vegetarian dish

Spicy $\sqcap$ VegetarianDish



Disjunction

Notation: $C \sqcup D$

Meaning: class of entities that belong to **either C or D (possibly both)**

Cat or dog

$\text{Cat} \sqcup \text{Dog}$

Cake or cookie or muffin

$\text{Cake} \sqcup (\text{Cookie} \sqcup \text{Muffin})$



Negation

Notation: $\neg C$

Meaning: class of entities that are **not in C**

Anything/anyone who is **not a parent**

$\neg \text{Parent}$

Person who is not a parent

$\text{Person} \sqcap (\neg \text{Parent})$



Existential Restrictions

Notation: $\exists r.C$ (r possibly complex role, C possibly complex concept)

Meaning: class of entities that **are related by r to an entity in C**

Those who teach a Master-level course offered by a French university

$\exists \text{teaches} . (\text{MastersCourse} \sqcap \exists \text{ offeredBy} . \text{FrenchUniv})$

Those who teach (something)

$\exists \text{teaches} . T$

Here T designates the set of all things



Universal Restrictions

Notation: $\forall r.C$ (r possibly complex role, C possibly complex concept)

Meaning: class of entities that are **only related by r to entities from C**

Those who teach only Master's courses

$\forall \text{teaches.MastersCourse}$

Question: Do they necessarily teach something?

NO: those who don't teach trivially belong to the class

If we only want to include those who teach, need to specify this:

$(\forall \text{teaches.MastersCourse}) \sqcap (\exists \text{teaches.T})$



Number Restrictions

Notation: $\geq kr.C, \leq kr.C$ (r role, C concept, k non-negative integer)

Meaning: class of entities that **are connected via r to at least / at most k elements of C**

Those having at least 3 adult children

$\geq 3\text{hasChild.Adult}$

Things that have at most 5 ingredients

$\leq 5 \text{ hasIngredient.T}$



Nominals

Notation:

$\{a\}$

(a an individual name)

Meaning: class consisting **solely of a**

Canada, USA, or Mexico

$\{\text{canada}\} \sqcup \{\text{usa}\} \sqcup \{\text{mexico}\}$

Courses taught by Meghyn

$\text{Course} \sqcap \exists \text{taughtBy}.\{\text{meghyn}\}$



Top Concept and Bottom Concept

- *top concept* $\top$
 - It is sometimes useful to be able to make a statement about every individual
 - Example: ***everybody is either male or female.***

$$\top \sqsubseteq \text{Male} \sqcup \text{Female}$$

- *bottom concept* $\perp$
 - To express that, nobody can be both a parent and childless at the same time, we can declare the set of parents and the set of childless individuals to be disjoint

$$\text{Parent} \sqcap \text{Childless} \sqsubseteq \perp$$

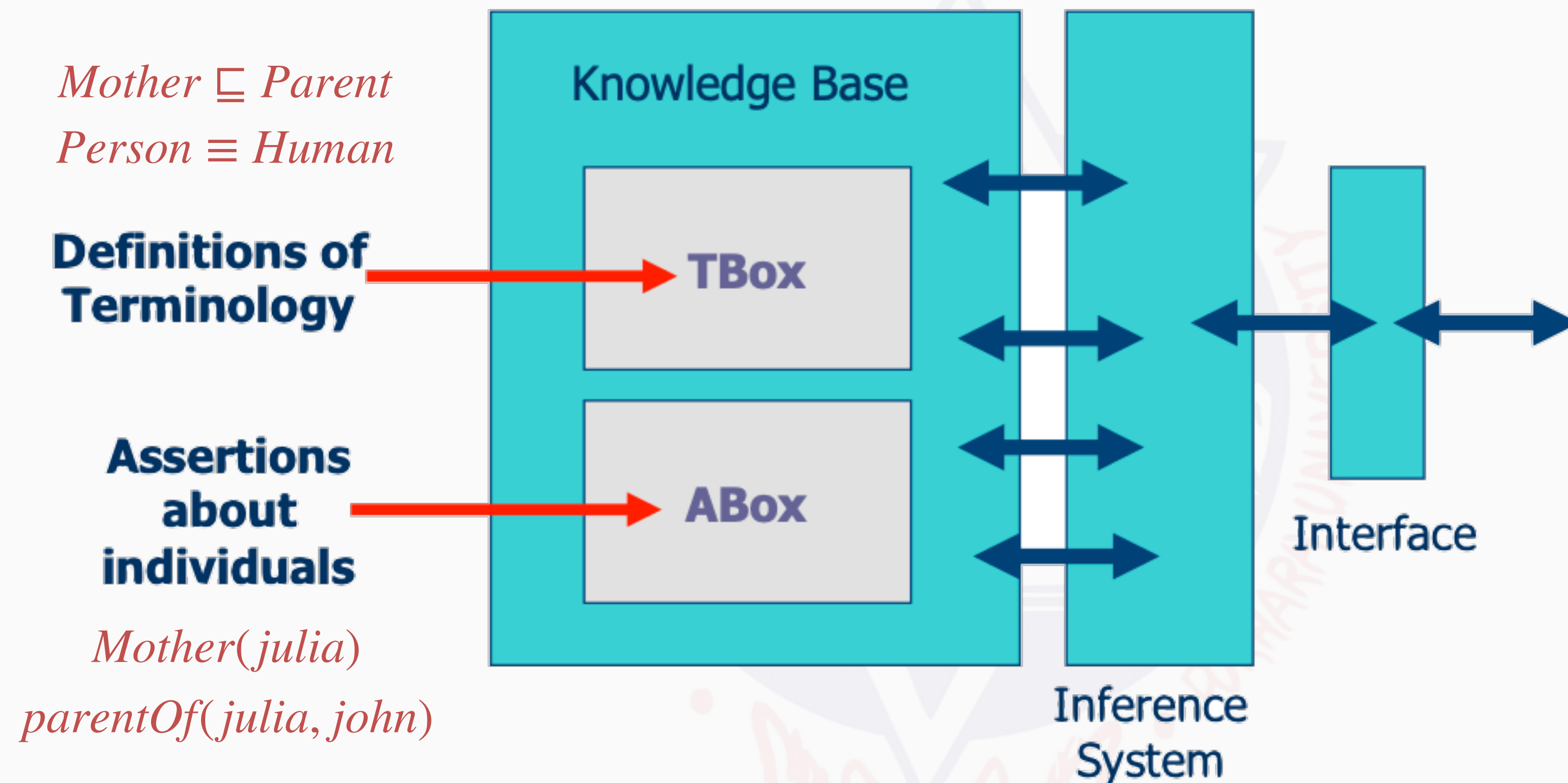


Description Logics

- DLs provide a logical reconstruction and (claimed to be a) unifying formalism for other knowledge representation languages, such as
 - frames-based systems
 - object-oriented modelling
 - Semantic data models, etc.
- They provide the language to formulate theories and systems declaratively expressing structured knowledge and for accessing it and reasoning with it, and they are used for, among others, terminologies and ontologies,



Components of Description Logics



The division into TBox and ABox doesn't have a logical significance, but is made for conceptual and implementation convenience.



Components of Description Logics

TBox (**Terminological Box**) is a set of axioms in the form of $(C \sqsubseteq D, C \doteq D)$ describing structure of domain (i.e., schema),

Example:

HappyFather $\doteq$ Man $\sqcap$ $\exists$ hasChild.Female

Elephant $\sqsubseteq$ Animal $\sqcap$ Large $\sqcap$ Grey

ABox (Assertion Box) is a set of axioms in the form of $(C(a), R(a, b))$ describing a concrete situation (data),

Example:

HappyFather (John)
hasChild(John, Mary)



Expressing Terminological Knowledge with TBox Axioms

- TBox axioms describe relationships between concepts.

- *concept inclusion*

- Fact: *all mothers are parents*

$$\textit{Mother} \sqsubseteq \textit{Parent}$$

in which case we say that the concept Mother is subsumed by the concept Parent

- *concept equivalence:*

$$\textit{Person} \equiv \textit{Human}$$

Concept equivalence asserts that two concepts have the same instances



Asserting Facts with ABox Axioms

- ABox axioms capture knowledge about named individuals, i.e., the concepts to which they belong and how they are related to each other.
- The most common ABox axioms are:

- *concept assertions:*

Mother(julia)

which asserts that Julia is a mother or, more precisely, that the individual named julia is an instance of the concept Mother.

- *role assertions:* (describe relations between named individuals)

parentOf(julia, john)

which states that *Julia* is a parent of *John* or, more precisely, that the individual named *julia* is in the relation that is denoted by parentOf to the individual named *john*



Modelling Relationships between Roles with RBox Axioms

- RBox axioms refer to properties of roles.
 - support *role inclusion* and *role equivalence* axioms.
- *role inclusion*:
$$\text{parentOf} \sqsubseteq \text{ancestorOf}$$

states that parentOf is a subrole of ancestorOf
i.e., every pair of individuals related by parentOf is also related by ancestorOf



Examples of TBox and ABox

Tbox:

$\text{Student} \doteq \text{Person} \sqcap \exists \text{NAME.String} \sqcap$
 $\exists \text{ADDRESS.String} \sqcap$
 $\exists \text{ENROLLED.Course}$
 $\exists \text{TEACHES.Course} \sqsubseteq \neg \text{Undergrad} \sqcap \text{Professor}$

Abox:

Student(Ali)
ENROLLED(Ali, Comp338)
(Student $\sqcup$ Professor)(Dima)



TBox Semantics

An interpretation I satisfies the statement $C \sqsubseteq D$ if $C^I \subseteq D^I$.

An interpretation I satisfies the statement $C \doteq D$ if $C^I = D^I$.

An interpretation I is a model for a TBox T if I satisfies all statements in T .



TBox Semantics

An interpretation I satisfies the statement $C \sqsubseteq D$ if $C^I \subseteq D^I$.

An interpretation I satisfies the statement $C \doteq D$ if $C^I = D^I$.

An interpretation I is a model for a TBox T if I satisfies all statements in T .



ABox Semantics

If $I = (\Delta^I, \cdot^I)$ is an interpretation,

$C(a)$ is satisfied by I if $a^I \in C^I$.

$R(a, b)$ is satisfied by I if $(a^I, b^I) \in R^I$.

A set A of assertions is called an ABox.

An interpretation I is said to be a *model* of the ABox A if every assertion of A is satisfied by I . The ABox A is said to be *satisfiable* if it admits a model.

An interpretation $I = (\Delta^I, \cdot^I)$ is said to be a *model of a knowledge base* Σ if every axiom of Σ is satisfied by I .

A knowledge base Σ is said to be *satisfiable* if it admits a model.



Syntax and semantics of SROIQ axioms

	<i>Syntax</i>	<i>Semantics</i>
<i>ABox:</i>		
concept assertion	$C(a)$	$a^I \in C^I$
role assertion	$R(a, b)$	$\langle a^I, b^I \rangle \in R^I$
individual equality	$a \approx b$	$a^I = b^I$
individual inequality	$a \not\approx b$	$a^I \neq b^I$
<i>TBox:</i>		
concept inclusion	$C \sqsubseteq D$	$C^I \subseteq D^I$
concept equivalence	$C \equiv D$	$C^I = D^I$
<i>RBox:</i>		
role inclusion	$R \sqsubseteq S$	$R^I \subseteq S^I$
role equivalence	$R \equiv S$	$R^I = S^I$
complex role inclusion	$R_1 \circ R_2 \sqsubseteq S$	$R_1^I \circ R_2^I \subseteq S^I$
role disjointness	$Disjoint(R, S)$	$R^I \cap S^I = \emptyset$



DLs

- **AL** (Attributive Language):
 - A basic logic supporting conjunction and limited existential quantification.
- **ALC**
 - Attributive Language with Concept negation
- **SHOIN**
 - Supports OWL DL features (basis for OWL 1).
- **SROIQ**
 - Adds role chains, qualified cardinality, nominals — basis for OWL 2 DL



DLs

Lots of DLs: differ on **which constructors and axioms allowed**

For example, the **prototypical expressive DL $\mathcal{ALC}$** is defined by:

- **concept constructors:** $\perp, \top, \neg, \sqcup, \sqcap, \exists r.C, \forall r.C$
- **no role constructors**
- only **concept inclusions** as axioms

The **lightweight DL $\mathcal{EL}$** is a fragment of $\mathcal{ALC}$:

- concept constructors **restricted to:** $\top, \sqcap, \exists r.C$

The **highly expressive DL $\mathcal{SHIQ}$** is defined by:

- all $\mathcal{ALC}$ concept constructors, **plus:** $\geq n r.C, \leq n r.C$
- **inverse roles (r^-)**
- concept inclusions, **role inclusions**, and **transitivity** axioms



AL: Attributive Language

Constructor	Syntax	Example
atomic concept	C	Human
<i>atomic</i> negation	$\sim C$	$\sim$ Human
atomic role	R	hasChild
conjunction	$C \wedge D$	Human $\wedge$ Male
value restriction	$R.C$	Human $\exists$ hasChild.Blond
existential rest. (lim)	$\exists R$	Human $\exists$ hasChild
Top (univ. conc.)	T	T
bottom (null conc)	$\perp$	$\perp$

for concepts C and D and role R



ALC

ALC

attributive language with complement
[Schmidt-Schauß&Smolka, 1991]

Naming scheme:

- basic language *AL*
- extended with **constructors** whose “letter” is added after the *AL*
- *C* stands for **complement**, i.e., *ALC* is obtained from *AL* by adding the complement ($\neg$) operator



Syntax of ALC

Let **C** and **R** be disjoint sets of **concept names** and **role names**, respectively.

ALC-concept descriptions are defined by induction:

- If $A \in \mathbf{C}$, then A is an **ALC**-concept description.
- If C, D are **ALC**-concept descriptions, and $r \in \mathbf{R}$, then the following are **ALC**-concept descriptions:
 - $C \sqcap D$ (conjunction)
 - $C \sqcup D$ (disjunction)
 - $\neg C$ (negation)
 - $\forall r.C$ (value restriction)
 - $\exists r.C$ (existential restriction)

Abbreviations:

- $\top := A \sqcup \neg A$ (top)
- $\perp := A \sqcap \neg A$ (bottom)
- $C \Rightarrow D := \neg C \sqcup D$ (implication)



Syntax of ALC

ALC is the smallest DL that is propositionally closed (i.e., includes full negation and disjunction) and include booleans (and, or, not) and restrictions on role values

constructor	Syntax	Example
atomic concept	C	Human
negation	$\sim C$	$\sim (\text{Human} \vee \text{Ape})$
atomic role	R	hasChild
conjunction	$C \wedge D$	Human $\wedge$ Male
disjunction	$C \vee D$	Nice $\vee$ Rich
value restrict.	$\exists R.C$	Human $\exists$ hasChild.Blond
existential rest.	$\exists R.C$	Human $\exists$ hasChild.Male
Top (univ. conc.)	T	T
bottom (null conc)	$\perp$	$\perp$



Semantics of ALC

An interpretation $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$ consists of a non-empty domain $\Delta^{\mathcal{I}}$ and an extension mapping $\cdot^{\mathcal{I}}$:

- $A^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}}$ for all $A \in \mathbf{C}$, concepts interpreted as sets
- $r^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}$ for all $r \in \mathbf{R}$. roles interpreted as binary relations

The extension mapping is extended to complex $\mathcal{ALC}$ -concept descriptions as follows:

- $(C \sqcap D)^{\mathcal{I}} := C^{\mathcal{I}} \cap D^{\mathcal{I}}$
- $(C \sqcup D)^{\mathcal{I}} := C^{\mathcal{I}} \cup D^{\mathcal{I}}$
- $(\neg C)^{\mathcal{I}} := \Delta^{\mathcal{I}} \setminus C^{\mathcal{I}}$
- $(\forall r.C)^{\mathcal{I}} := \{d \in \Delta^{\mathcal{I}} \mid \text{for all } e \in \Delta^{\mathcal{I}} : (d, e) \in r^{\mathcal{I}} \text{ implies } e \in C^{\mathcal{I}}\}$
- $(\exists r.C)^{\mathcal{I}} := \{d \in \Delta^{\mathcal{I}} \mid \text{there is } e \in \Delta^{\mathcal{I}} : (d, e) \in r^{\mathcal{I}} \text{ and } e \in C^{\mathcal{I}}\}$



Why DL?

- Suppose a KB containing:
 - "Every student in the university is supervised by a professor."
 - "Every professor may supervise any number of students, but only if they are employed by the university."
 - "Some people are students and professors at the same time."
- Problem: Does this knowledge base entail that there is at least one employed professor?



Why DL?

- Issues in the question?
 - To answer this, a system must reason through all possible combinations of students, professors, and supervisors, including those who are self-supervised (since someone could be both student and professor).
 - It must consider infinite models: What if there are infinitely many universities or people?
 - There is no guarantee that an automated system can finish checking all the possibilities — the search space is infinite.
 - Therefore, no algorithm can decide this in general, for all FOL knowledge bases.



Why DL?

- Why this happens?
 - The expressive power of FOL lets us:
 - Quantify over infinite sets
 - Define recursive properties
 - Express complex constraints (e.g., supervisor chains)
- But this power leads to undecidability —
 - there's no general algorithm that can always decide whether a conclusion logically follows from a set of FOL statements.





THANK YOU